



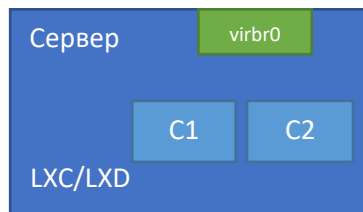
Управление инфраструктурой IT-проекта

Задание 3. LXC/LXD

Познакомимся с контейнеризацией в Ubuntu.

Этап 1. Установка LXC.

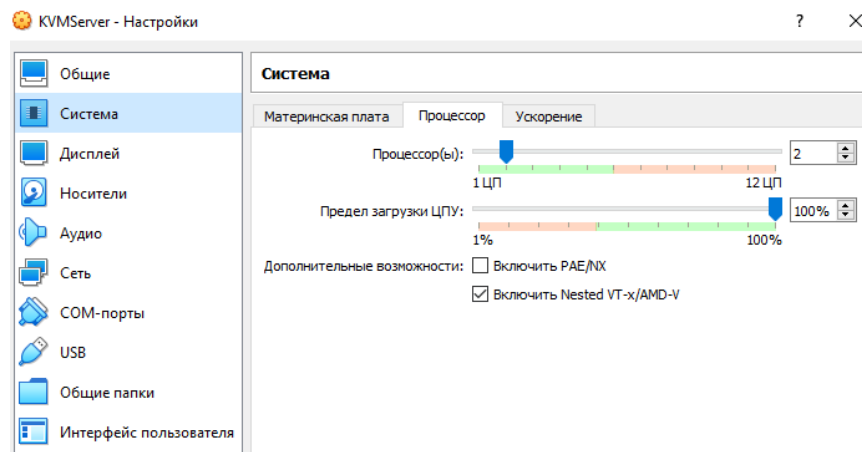
Для этой части нам понадобится инфраструктура, подобная этой:



Создайте виртуальную машину для сервера, импортировав шаблон и выдав 2 ядра и 4гб оперативки (назовите LXCServer):

Виртуальная система 1	
Имя	KVMServer
Тип гостевой ОС	Ubuntu (64-bit)
Процессор	2
ОЗУ	4096 МБ
DVD-привод	☒
USB-контроллер	☒
Звуковая карта	☒ ICH AC97
Сетевой адаптер	☒ Intel PRO/1000 MT Desktop (82540EM)
Контроллер (IDE)	PIIX4
Контроллер (IDE)	PIIX4
Контроллер (SATA)	AHCI
Виртуальный образ диска	Ubuntu-Template-disk001.vmdk
Базовый каталог	C:\Users\LinWin\VirtualBox VMs
Основная группа	/

Зайдите в настройки созданной машины и включите аппаратную виртуализацию (Nested VT-x/AMD-V):





Управление инфраструктурой IT-проекта

Запускаем машину в фоновом режиме (без экрана), подключаемся через PuTTY, логинемся.

Сделаем снэпшот для сохранения состояния машины.

В Ubuntu мы можем установить LXC без какой-либо подготовки:

```
sudo apt-get update
```

```
sudo apt-get install lxc lxc-templates
```

Далее запустим демон, если он еще не запущен

```
sudo systemctl enable --now lxc
```

Теперь мы можем проверить конфигурацию:

```
lxc-checkconfig
```

Изучите папку шаблонов

```
ls -l /usr/share/lxc/templates/
```

Здесь вы увидите шаблоны для большинства популярных дистрибутивов Linux.

Шаблон, который мы будем использовать в основном – **lxc-download**.

Этап 2. Настройка сети для LXC.

В Ubuntu сетевое подключение и инфраструктура для контейнеров автоматически управляется LXC.

Изучите файл `/etc/lxc/default.conf`, он содержит базовую конфигурацию, связанную с сетью.

```
sudo cat /etc/lxc/default.conf
```

Этап 3. Создание контейнеров LXC.

Давайте создадим наш первый контейнер.

Мы должны заранее знать, каков наш выбор дистрибутива, релиза дистрибутива (версии) и аппаратной архитектуры.

Доступные шаблоны можно глянуть тут: <https://images.linuxcontainers.org/>.

Предположим, что мы этого не знаем, и поэтому мы хотим создать контейнер интерактивным способом.

Воспользуемся командой:

```
sudo lxc-create -t download -n con1
```

Вы можете получить уведомление о том, что пакет либо `tar`, либо `wget` отсутствует, тогда установите их.



Управление инфраструктурой IT-проекта

Если вы получите уведомление с сообщением об ошибке «ERROR: Unable to fetch GPG key from keyserver», то изменить приведенную выше команду на:

```
sudo lxc-create -t download -n con1 -- --no-validate
```

В интерактивной форме нам предоставят варианты ответов.

Давайте выберем alpine в качестве дистрибутива, версии 3.14 или выше, и соответствующую архитектуру (amd64 для современных AMD и Intel процессоров).

Через некоторое время контейнер будет доступен на вашей машине.

Теперь, когда мы знаем, какие у нас есть варианты, давайте создадим второй контейнер того же типа, но на этот раз с помощью другой команды, более расширенной:

```
sudo lxc-create --template download --name con2 -- --no-validate --  
dist alpine --release 3.14 --arch amd64
```

Этап 4. Работа с контейнерами.

Давайте запустим наш первый контейнер, выполним:

```
sudo lxc-start --name con1
```

Чтобы проверить, какие контейнеры существуют в нашей системе и каково их состояние:

```
sudo lxc-ls --fancy
```

Теперь мы можем видеть, какой из них запущен, каков его IP-адрес и так далее...

Хорошо, теперь, когда мы знаем IP-адрес нашего контейнера, давайте попробуем подключиться через SSH!

И... ничего не получится 😞

Подключимся к контейнеру:

```
sudo lxc-attach --name con1
```

Теперь, когда мы вошли в shell контейнера, мы можем выполнить любую произвольную команду, такую как ip, uname, hostnamectl и т.д.

Давайте проверим, есть ли у нас подключение к Интернету изнутри:

```
ping ya.ru
```

Можем ли мы создавать файлы? Попробуем. Создайте простой файл **/readme.txt**:

```
echo "Hello world!" > /readme.txt
```

Проверьте, есть ли файл, и можете ли вы прочитать его содержимое? Используйте команду ls и cat.

Давайте проверим, установлен ли OpenSSH и работает ли он?

```
rc-service sshd describe
```



Установим тогда:

```
apk add openssh
```

И запустим:

```
rc-service sshd start
```

```
rc-update add sshd
```

И закройте соединение с контейнером:

```
exit
```

Подумали, что вы уничтожили контейнер? Давайте проверим!

```
sudo lxc-ls --fancy
```

Давайте попробуем установить SSH соединение:

```
ssh root@<container-ip>
```

Похоже, что мы не знаем пароля ...

Конечно, мы можем подключиться опять в shell для контейнера и установить его, но давайте рассмотрим другой подход!

Этап 5. Работа с контейнерами ч.2.

Давайте рассмотрим контейнер поближе. Запросите системную информацию о контейнере:

```
sudo lxc-info --name con1
```

Ладно, мы увидели кое-какую информацию, но где хранится этот контейнер?

Давайте проверим, какие параметры конфигурации у нас есть:

```
sudo lxc-config -l
```

Узнаем у lxc.lxcpath:

```
sudo lxc-config lxc.lxcpath
```

Теперь давайте посетим эту папку и изучим ее содержимое

```
ls -al /var/lib/lxc
```

Похоже, что у каждого нашего контейнера здесь есть своя папка

Давайте проверим, каково содержимое папки con1

```
sudo ls -al /var/lib/lxc/con1
```

Есть файл с именем config и другая папка rootfs

Не должно вызывать удивления, что конфигурационный файл содержит конфигурацию для контейнера:



Управление инфраструктурой IT-проекта

```
sudo cat /var/lib/lxc/con1/config
```

Давайте проверим папку rootfs:

```
sudo ls -al /var/lib/lxc/con1/rootfs
```

Опа, даже readme.txt файл имеется. Можем ли мы его прочитать?

```
sudo cat /var/lib/lxc/con1/rootfs/readme.txt
```

Конечно, мы можем. Но можем ли мы написать что-то в него?

```
echo "Changed by the host" | sudo tee -a  
/var/lib/lxc/con1/rootfs/readme.txt
```

Очевидно, что мы можем!

Можем ли мы изменить или установить пароль пользователя root для контейнера?

Мы можем использовать метод chroot для установки пароля:

```
sudo chroot /var/lib/lxc/con1/rootfs/ passwd
```

Теперь, когда у нас есть пароль, давайте попробуем подключиться по SSH

```
ssh root@<container-ip>
```

Снова не повезло. Возможно, root'у запрещено использовать аутентификацию по паролю через SSH

Давайте вернемся обратно к контейнеру

```
sudo lxc-attach --name con1
```

Проверьте и измените конфигурацию SSH:

```
vi /etc/ssh/sshd_config
```

Добавьте следующее, сохраните и закройте файл

(<https://serverspace.ru/support/help/shpargalka-po-vim/> как пользоваться vim)

```
PermitRootLogin yes
```

Перезапустите службу

```
rc-service sshd restart
```

Выходим из контейнера

```
exit
```

Попробуйте еще раз

```
ssh root@<container-ip>
```

Вуаля, наконец-то мы можем установить SSH-сеанс между хостом и контейнером!



Управление инфраструктурой IT-проекта

Пока вы здесь, проверьте /readme.txt файл и убедитесь, что изменения были совершены (командой cat)

Закроем соединение:

exit

Этап 6. Остановка, запуск, автозапуск контейнеров.

Мы можем остановить контейнер как внутри, так и внешне.

К примеру, внутри контейнера введите:

poweroff

Контейнер остановлен. Мы можем проверить:

sudo lxc-ls --fancy

Давайте стартанем его снова:

sudo lxc-start --name con1

Давайте выполним альтернативный вариант команды lxc-attach:

sudo lxc-attach --name con1 -- uname -a

Как вы смогли догадаться, вы можете передавать команды таким образом. Давайте попробуем отключить контейнер, передавая команды:

sudo lxc-attach --name con1 -- poweroff

Отлично!

Давайте стартанем его снова:

sudo lxc-start --name con1

И попробуйте еще один альтернативный подход для остановки контейнера:

sudo lxc-stop --name con1

Если мы захотим убить его вместо того, чтобы мягко остановить, мы можем выполнить команду:

sudo lxc-stop --name con1 --kill

Если мы заменим команду --kill на --reboot, мы перезапустим контейнер

Мы можем настроить контейнер на автоматический запуск, установив параметру lxc.start.auto значение 1 в файле конфигурации контейнера, внесите изменение в файл и сохраните:

sudo nano /var/lib/lxc/con1/config

Затем мы можем попросить все контейнеры, настроенные на автозапуск, стартовать:

sudo lxc-autostart



Этап 6. Клонирование контейнеров.

Мы можем клонировать остановленные контейнеры!

Давайте клонируем наш контейнер con2 в con3

```
sudo lxc-copy --name con2 --newname con3
```

Проверим список контейнеров:

```
sudo lxc-ls --fancy
```

Иногда нам нужно быстро создать тестовый клон для простого теста. Мы можем создавать для этого связанные клоны. Давайте создадим один:

```
sudo lxc-copy --name con2 --newname con2-test -B overlay -s
```

Это происходит быстрее, чем традиционное клонирование.

Давайте проверим, что происходит на сервере с хранилищем. Сравните папки con2 и con2-test:

```
sudo ls -al /var/lib/lxc/con2/
```

```
sudo ls -al /var/lib/lxc/con2-test/
```

Появилась дополнительная папка delta0, которая содержит изменения по сравнению с исходным контейнером (связанным). К примеру, имя хоста:

```
sudo ls -al /var/lib/lxc/con2-test/delta0/etc
```

Этап 7. Снапшоты.

Мы также можем создать моментальный снимок (снапшот). Опять же, это должно быть сделано на остановленных контейнерах (поэтому, если контейнер запущен, сначала остановите его).

Давайте создадим снимок нашего первого контейнера.

Если мы хотим прикрепить комментарий к снимку, мы должны создать текстовый файл с ним:

```
echo "snapshot 1" > snapshot-comment
```

А затем создадим сам снимок с комментарием

```
sudo lxc-snapshot --name con1 --comment snapshot-comment
```

Мы можем получить список моментальных снимков контейнера с помощью:

```
sudo lxc-snapshot --name con1 --list
```

Чтобы также увидеть комментарии:

```
sudo lxc-snapshot --name con1 --list --showcomments
```

Мы можем восстановить или вернуть состояние контейнера к снимку:



Управление инфраструктурой IT-проекта

```
sudo lxc-snapshot --name con1 --restore snap0
```

В качестве альтернативы мы можем восстановить снимок в виде нового контейнера:

```
sudo lxc-snapshot --name con1 --restore snap0 --newname con1-snap
```

Этап 7. Мониторинг.

Мы можем отслеживать, что происходит с контейнерами:

```
sudo lxc-top
```

Чтобы закрыть, нажмите клавишу Q.

Этап 8. Удаление контейнеров.

Мы можем удалить контейнер:

```
sudo lxc-destroy --name con2-test
```

Если в контейнере есть моментальные снимки, мы можем удалить их вместе с контейнером

```
sudo lxc-destroy --name con1 --snapshots
```

Удалите все остальные контейнеры ☺

Этап 9. Установка LXD.

LXD — это усовершенствованный подход, по сравнению с классическим LXC.

Чтобы установить его, выполните:

```
sudo snap install lxd
```

Через некоторое время все будет установлено и готово!

Переключитесь на пользователя «root»

```
sudo su
```

Создайте начальную конфигурацию LXD:

```
lxd init
```

Следуйте инструкциям, и как только закончите, вернитесь к обычному пользователю (указанного при установке).

Проверьте, можете ли вы выполнять команды LXD. Проверьте версию как клиента, так и сервера:

```
lxc version
```

Подробная информация

```
lxc info
```

Этап 10. Создание контейнеров и работа с ними.



Управление инфраструктурой IT-проекта

Давайте сначала попробуем выполнить поиск образов:

```
lxc image list images:
```

Список довольно длинный! Давайте немного отфильтруем:

```
lxc image list images: | grep -i ubuntu
```

Хорошо, запустите контейнер Ubuntu Focal:

```
lxc launch images:ubuntu/focal/amd64 focal-con
```

И еще один Ubuntu Bionic:

```
lxc launch images:ubuntu/bionic/amd64 bionic-con
```

Теперь проверьте, какие локальные образа у нас есть:

```
lxc image list
```

Мы можем проверить запущенные контейнеры:

```
lxc list
```

Мы можем управлять контейнерами через обычные действия "stop", "restart" и "start". Например, перезапустите контейнер Focal и остановите, а затем запустите Bionic:

```
lxc restart focal-con
```

```
lxc stop bionic-con && lxc start bionic-con
```

Чтобы получить более подробную информацию о контейнере, выполните:

```
lxc info focal-con
```

Мы можем выполнять команды в контейнерах с хоста. Например:

```
lxc exec focal-con uname
```

Если нам нужно указать параметры, мы должны изменить команду:

```
lxc exec focal-con -- uname -a
```

Если нам нужно установить сеанс с контейнером и работать там, мы можем выполнить:

```
lxc exec focal-con bash
```

По сути, вы можете выполнять команды так, как если бы это был обычный хост.

Этап 11. Удаление контейнеров.

Остановленные контейнеры, можем удалить командой:

```
lxc delete focal-con
```

Или, если контейнер запущен, мы можем изменить команду на:

```
lxc delete --force focal-con
```